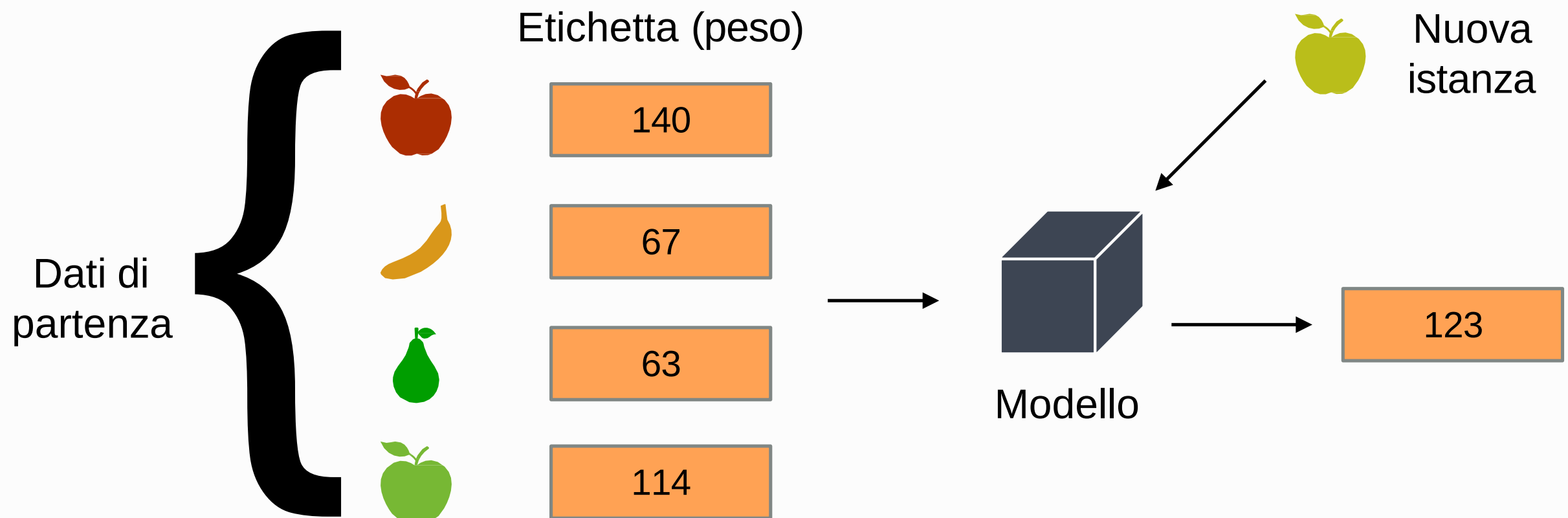


Regressione, Regressori, e Oltre

Problemi di regressione

Regressione: Task in cui l'obiettivo è predire il valore di una variabile numerica, chiamata variabile dipendente o di risposta, tramite l'utilizzo di un training set, ovvero un insieme di osservazioni per cui la variabile dipendente è nota.

I problemi di regressione sono istanze di problemi di apprendimento supervisionato.



Problemi di regressione

Regressione: Task in cui l'obiettivo è predire il valore di una variabile numerica, chiamata variabile dipendente o di risposta, tramite l'utilizzo di un training set, ovvero un insieme di osservazioni per cui la variabile dipendente è nota.

I problemi di regressione sono istanze di problemi di apprendimento supervisionato.

In maniera simile alla classificazione, un problema di regressione porta alla costruzione di un *modello*, ovvero di uno strumento che fa uso di un algoritmo di apprendimento, anche detto *regressore*, per predire i nuovi elementi sulla base del training set.

I regressori sono essenzialmente delle funzioni matematiche che cercano di descrivere i dati. Diversi regressori si distinguono tra di loro per via delle assunzioni fatte sui dati così come delle specifiche proprietà che portano alla regressione, ma anche del numero di variabili indipendenti (predittori) di cui disponiamo.

Come fatto per i problemi di classificazione, approfondiremo due regressori in particolare: la *regressione singola/multipla* e gli *alberi decisionali*. Più importante, però discuteremo come è possibile selezionare il *giusto* regressore in base al problema.

Anche per i problemi di regressione, è possibile parlare di *ensemble learning*, ovvero della combinazione di più regressori. Tuttavia, queste tecniche sono più complesse di quelle usate in classificazione, poiché non possono limitarsi a giudicare le classi generate, ma devono interpretare i valori numerici predetti da più modelli.

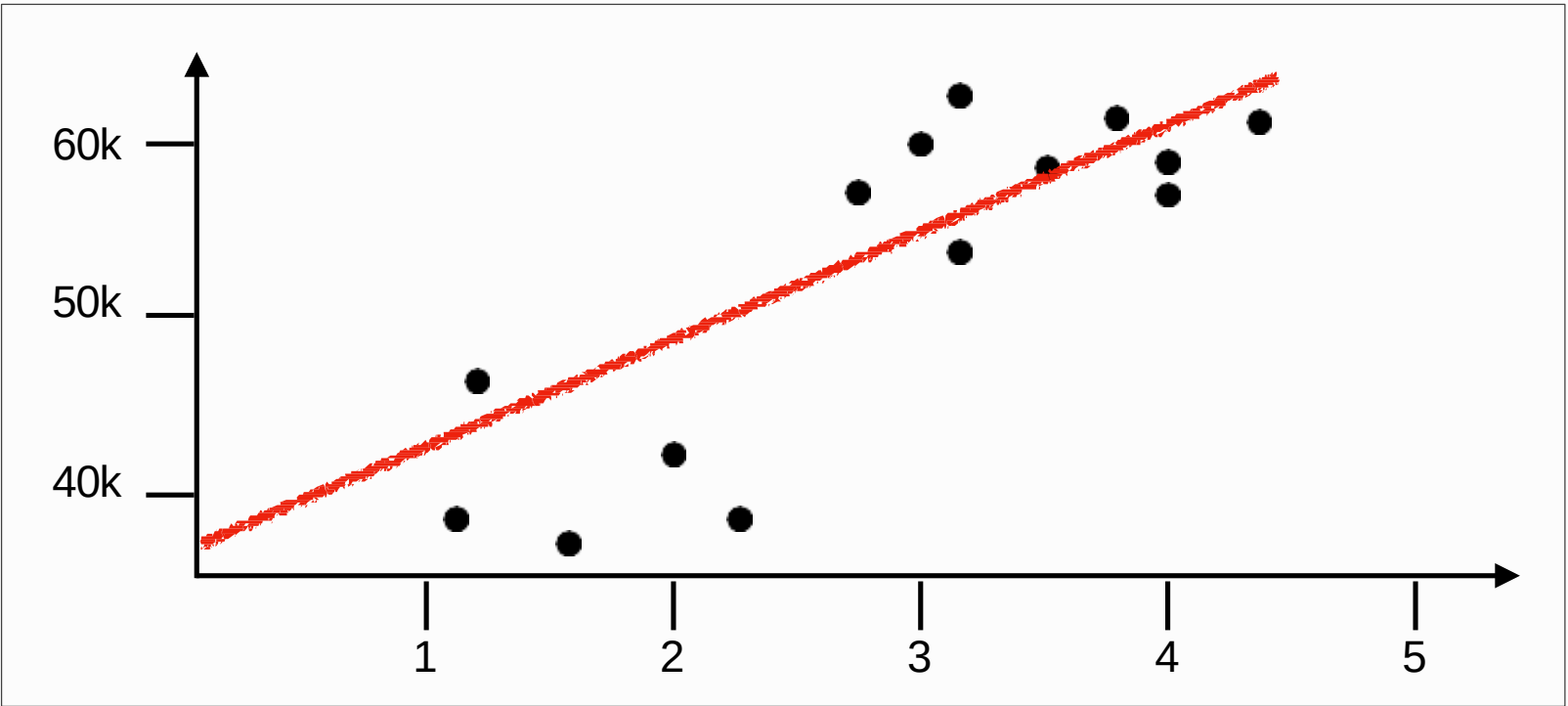
Regressione lineare singola e multipla

La differenza tra modelli singoli e multipli dipende dal numero di predittori che abbiamo a disposizione. Questo avrà un impatto sulla funzione che verrà generata.

Salario	Anni di esperienza
39.343	1,1
46.205	1,3
37.731	1,5
43.525	2,0
39.891	2,2
56.655	2,9
60.150	3,0
54.445	3,2
64.313	3,2
57.189	3,7
63.218	3,9
55.794	4,0
56.957	4,0
61.111	4,5

Supponiamo di dover predire il salario di un impiegato sulla base dei suoi anni di esperienza.

Questo è un problema risolvibile con una regressione lineare singola, in quanto abbiamo un unico predittore.



$$\tilde{y} = 32.375,85 + 6.995,31 \cdot \text{anniDiEsperienza}$$

La funzione verrà poi usata per predire nuovi dati.

Regressione, Regressori, e Oltre

Regressione lineare singola e multipla

Il punto quindi è quello di trovare una funzione che si adatti ai dati di training, cosicché possa essere efficace per predire il valore della variabile dipendente su nuovi dati.

In termini più pratici, generalizzando la funzione di regressione lineare, avremo:

$$\tilde{y} = \beta_0 + \beta_1 \cdot x_1$$

Questo indicherà
l'inclinazione della retta

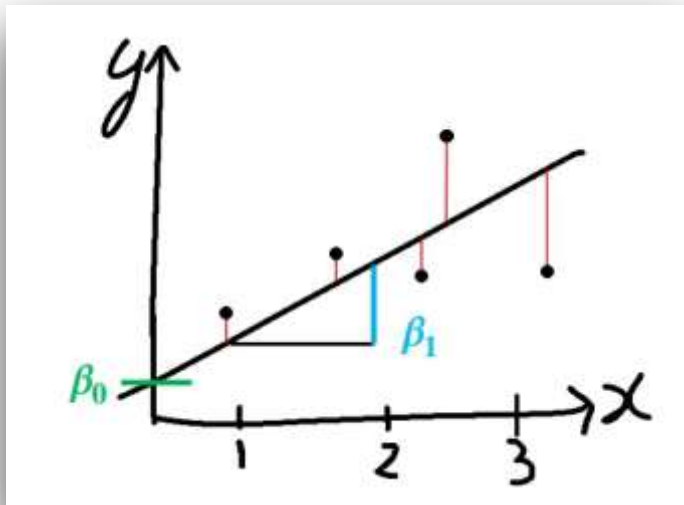
Per i più smemorati, l'intercetta β_0 indica il valore di y_i quando x_i è zero.

L'inclinazione indica la variazione di y_i quando x_i incrementa di una unità.

$$y = \tilde{y} + \epsilon$$

Questo indicherà
il residuo

Il residuo è la differenza tra il valore reale ed il valore predetto dal modello di regressione.



Se la definizione di regressione lineare singola è abbastanza semplice, meno semplice è capire quando una retta si adatta bene ai dati di training.

Regressione, Regressori, e Oltre

Regressione lineare singola e multipla

A questo proposito, occorre nuovamente parlare di sperimentazione empirica e di metriche di valutazione.

Mean Absolute Error (MAE)

$$\frac{\sum_{i=1}^n |\tilde{y} - y|}{n}$$

La metrica MAE indica la differenza media osservata tra i valori predetti e i valori reali del test set.

Mean Squared Error (MSE)

$$\frac{\sum_{i=1}^n (\tilde{y} - y)^2}{n}$$

La metrica MSE indica l'errore quadratico medio commesso sui dati presenti nel test set.

Root Mean Squared Error (RMSE)

$$\sqrt{\frac{\sum_{i=1}^n (\tilde{y} - y)^2}{n}}$$

La metrica RMSE indica la radice quadrata dell'errore quadratico medio commesso sui dati presenti nel test set.

Regressione, Regressori, e Oltre

Regressione lineare singola e multipla

Il punto quindi è quello di trovare una funzione che si adatti ai dati di training, cosicché possa essere efficace per predire il valore della variabile dipendente su nuovi dati.

In termini più pratici, generalizzando la funzione di regressione lineare, avremo:

$$\tilde{y} = \beta_0 + \beta_1 \cdot x_1$$

$$y = \tilde{y} + \epsilon \quad \longrightarrow \quad \epsilon = y - \tilde{y}$$

I modelli di regressione puntano a trovare la retta che si adatta meglio ai dati. Il più semplice criterio è quello della minimizzazione del residuo: più basso è il residuo, più la retta sarà vicina ai dati.

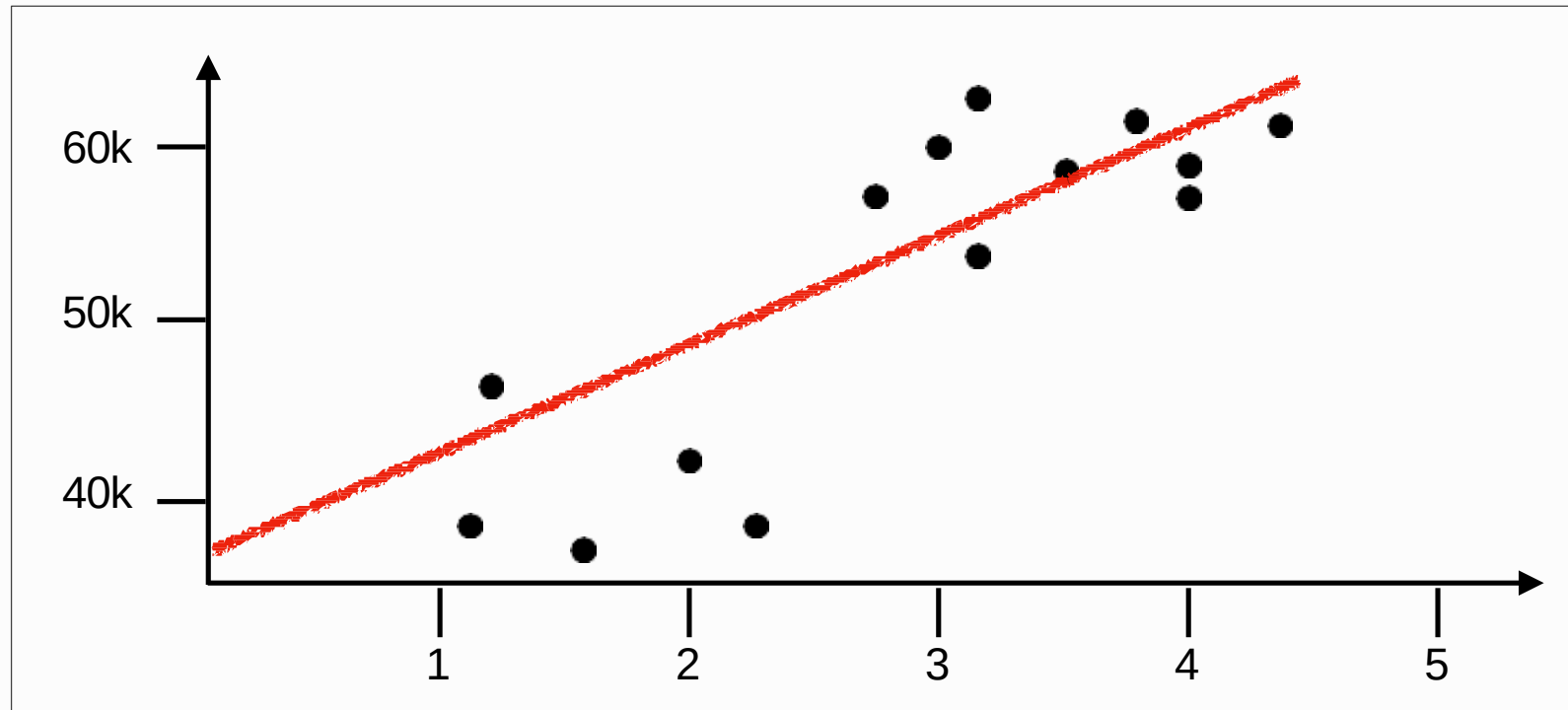
La somma dei residui è detta *funzione di perdita* (loss function). Molte delle tecniche di regressione mirano a minimizzare la funzione di perdita.

Calcolare i parametri della funzione

β_0 Il metodo più conosciuto è quello dei minimi quadrati (least squared), che avete visto o vedrete più nel dettaglio nei corsi di Calcolo Scientifico e Statistica ed Analisi dei Dati.

β_1 L'idea dietro al calcolo è relativamente semplice ed intuitiva.

Regressione lineare singola e multipla



1. Per ogni punto noto (x, y) dell'insieme N di punti, calcolare x^2 e xy .
2. Sommare tutti gli x , y , x^2 , e xy . Questo ci porta ad avere: $\sum x$, $\sum y$, $\sum x^2$, $\sum xy$;
3. Calcolare l'inclinazione della retta, tramite la formula:

$$\beta_1 = \frac{N \cdot \sum xy - \sum x \cdot \sum y}{N \cdot \sum (x^2) - (\sum x)^2}$$

4. Calcolare l'intercetta della retta, tramite la formula: $\beta_0 = \frac{\sum y - \beta_1 \cdot \sum x}{N}$
5. Mettere tutto insieme nella versione: $\tilde{y} = \beta_0 + \beta_1 \cdot x_1$

Regressione lineare singola e multipla

Sebbene sia semplice e talvolta efficace, la regressione lineare può essere utilizzata solo in determinati contesti. In particolare, il suo utilizzo assume:

Linearità dei dati. La relazione tra variabile indipendente X e variabile dipendente Y deve essere lineare, ovvero può essere rappresentata tramite una funzione lineare.

Normalità dei residui. Gli errori residui devono essere normalmente distribuiti.

Omoschedasticità. Gli errori residui devono avere una varianza costante.

Indipendenza degli errori. Gli errori residui devono essere indipendenti per ogni valore di X . Un test statistico particolarmente utile è noto come *Durbin-Watson*: quando gli errori sono indipendenti, il valore del test sarà vicino a 2.



Quando si parla di statistica, il linguaggio di programmazione ed ambiente di sviluppo R è quello più appropriato, poiché implementa già molte delle funzioni e dei test menzionati.

Giusto come esempio, il modello di esempio sarebbe implementato come:

```
regressore <- lm(salario~anniDiEsperienza, data=dataset)
```

Una guida per l'implementazione della regressione in R è disponibile qui: <https://www.scribbr.com/statistics/linear-regression-in-r/>

Regressione, Regressori, e Oltre

Regressione lineare singola e multipla

Ma cosa succede se non ho un'unica variabile indipendente? In questo caso, parliamo di regressione lineare multipla.

$$\tilde{y} = \beta_0 + \beta_1 \cdot x_1 + \beta_2 \cdot x_2 + \dots + \beta_m \cdot x_m$$

$$y = \beta_0 + \beta_1 \cdot x_1 + \beta_2 \cdot x_2 + \dots + \beta_m \cdot x_m + \epsilon$$

I parametri rappresentano l'effetto che la variazione di un'unità delle variabili indipendenti hanno sulla variabile dipendente

Come nel caso precedente, il metodo dei minimi quadrati rappresenta il modello più semplice di risoluzione di un problema di regressione multipla.

La differenza sta nel fatto che verrà utilizzata una forma matriciale per rappresentare le variabili dipendenti ed indipendenti, oltre che non vorremo più cercare una singola retta di regressione, ma un piano che meglio interpola i dati di training.

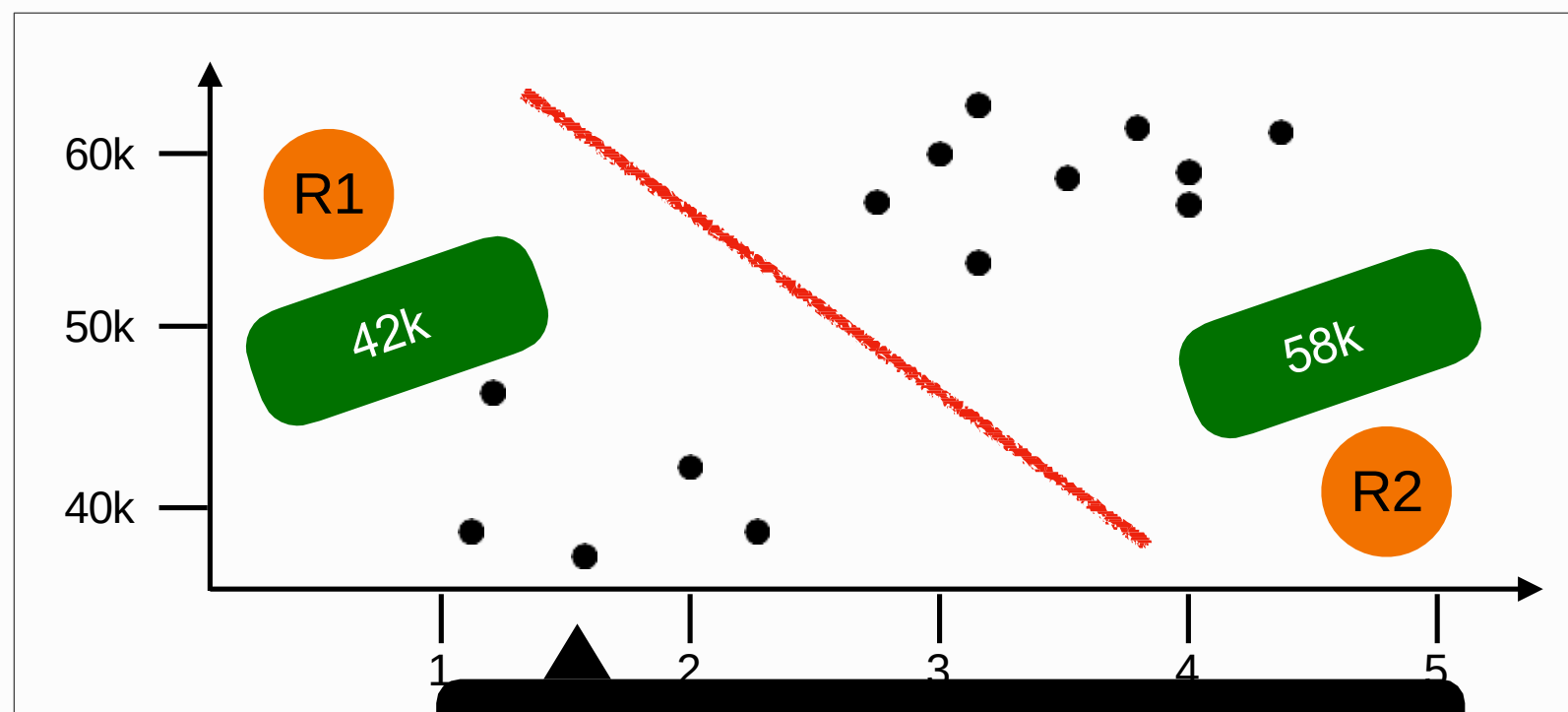
La cosa importante da sapere è che per la regressione lineare multipla valgono gli stessi vincoli di linearità e omoschedasticità della regressione lineare singola.

Altra cosa fondamentale: avendo più variabili, potremmo ricadere nel problema della multicollinearità —> è perciò importante eliminare le variabili ridondanti!

Alberi decisionali regressivi

Come secondo caso di studio, consideriamo gli alberi *regressivi*, ovvero la variante degli alberi decisionali utilizzabile per problemi di regressione.

L'idea alla base degli alberi regressivi è quella di (1) individuare delle regioni dello spazio nelle quali i valori interni assumono valori simili e (2) calcolare un valore rappresentativo di tali regioni; e (3) assegnare i nuovi dati alla regione più plausibile.



In altri termini, facendo riferimento all'esempio precedente, vogliamo identificare i confini (la linea rossa in figura) che delimitano delle regioni dello spazio (R1 ed R2 in figura) in cui i valori assumono valori simili.

In secondo luogo, vogliamo identificare un valore rappresentativo per ogni regione dello spazio, ad esempio la media.

Sulla base di questi valori, l'albero decisionale sarà capace di sfruttare le feature per decidere a quale regione assegnare un nuovo valore da predire.

Alberi decisionali regressivi

Albero decisionale: L'algoritmo mira a creare un albero i cui nodi rappresentano un sotto-insieme di caratteristiche del problema e i cui archi rappresentano delle decisioni.

L'algoritmo di un albero decisionale:

- (1) Posiziona la miglior caratteristica del training set come radice dell'albero;

Questa è rappresentata dalla caratteristica che suddividerà il set in modo che la somiglianza dei valori target sia massimizzata. Questa variabile è chiamata variabile di split.

Mentre negli alberi di classificazione si utilizza l'Information Gain, qui calcoleremo, per ogni caratteristica, la **varianza** dei valori della variabile dipendente. Quindi:

Varianza (Var)

$$\frac{\sum (X - \mu)^2}{N}$$

Misura della variabilità dei valori assunti dalla variabile dipendente dato il sottoinsieme di istanze corrispondente ad un valore della variabile indipendente.

La varianza totale di una variabile indipendente è data dalla somma delle varianze calcolate per ogni valore della variabile indipendente.

Alberi decisionali regressivi

Albero decisionale: L'algoritmo mira a creare un albero i cui nodi rappresentano un sotto-insieme di caratteristiche del problema e i cui archi rappresentano delle decisioni.

L'algoritmo di un albero decisionale:

(1) Posiziona la miglior caratteristica del training set come radice dell'albero;

Questa è rappresentata dalla caratteristica che suddividerà il set in modo che la somiglianza dei valori target sia massimizzata. Questa variabile è chiamata variabile di split.

(2) Calcola il miglior punto di split;

Per la variabile di split selezionata, calcola il punto che massimizza la riduzione della varianza. L'obiettivo è ridurre la varianza del target all'interno di ciascun gruppo creato dallo split.

Per una variabile continua, ordinare in modo crescente tutti i valori unici della variabile all'interno del set di dati e calcolare i punti di split potenziali tra i valori ordinati. Ad esempio, se avessimo i valori [1, 3, 5, 7], i potenziali punti di split potrebbero essere tra 1 e 3, tra 3 e 5, e tra 5 e 7. Per ciascun di questi, calcola la riduzione della varianza che si otterrebbe dividendo il set di dati in base a quel punto di split.

Per una variabile discreta, i punti di split sono rappresentati dai valori che può assumere la variabile. Scegliamo il valore che minimizza la varianza della variabile dipendente.

Alberi decisionali regressivi

Albero decisionale: L'algoritmo mira a creare un albero i cui nodi rappresentano un sotto-insieme di caratteristiche del problema e i cui archi rappresentano delle decisioni.

L'algoritmo di un albero decisionale:

(1) Posiziona la miglior caratteristica del training set come radice dell'albero;

Questa è rappresentata dalla caratteristica che suddividerà il set in modo che la somiglianza dei valori target sia massimizzata. Questa variabile è chiamata variabile di split.

(2) Calcola il miglior punto di split;

Per la variabile di split selezionata, calcola il punto che massimizza la riduzione della varianza. L'obiettivo è ridurre la varianza del target all'interno di ciascun gruppo creato dallo split.

(3) Suddivisione del dataset;

Dividi il set di dati in base alla variabile di split e al punto di split calcolato.

(4) Ripeti gli step (1), (2) e (3) su ogni sotto-insieme fin quando non viene raggiunto un criterio di fermata.

I criteri di fermata possono includere la profondità massima dell'albero, un numero minimo di campioni in una foglia o quando si raggiungono set di dati completamente omogenei.

Alberi decisionali regressivi

Albero decisionale: L'algoritmo mira a creare un albero i cui nodi rappresentano un sotto-insieme di caratteristiche del problema e i cui archi rappresentano delle decisioni.

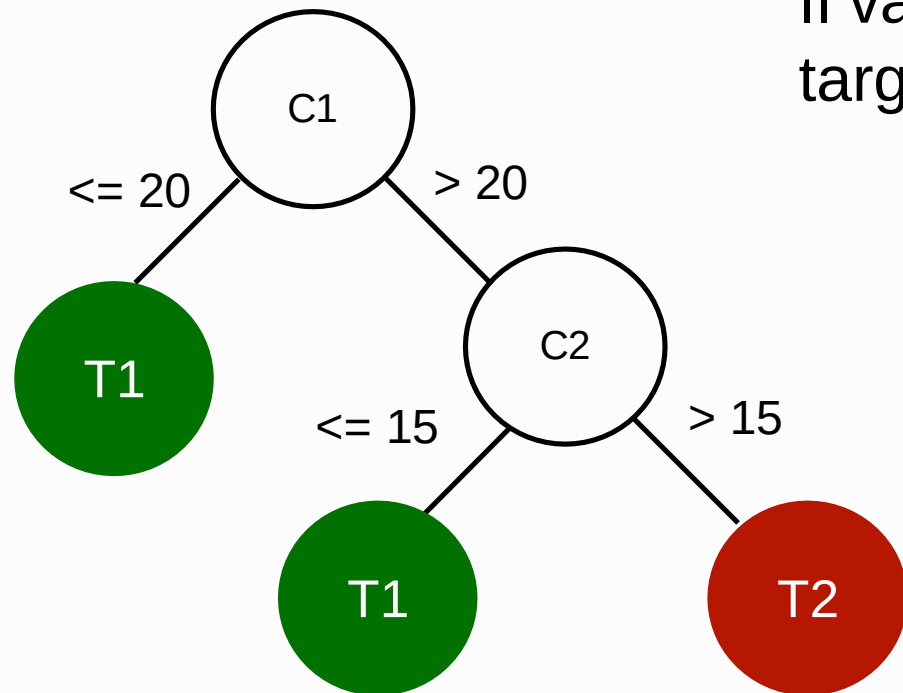
Ultimo problema. Dopo aver diviso il set di dati sulla base della varianza, avremo un albero decisionale. Ma quali valori assegniamo alle foglie?

Il valore di output di una foglia è spesso la media dei valori target delle istanze di dati che raggiungono quella foglia.

Ad esempio, se stiamo prevedendo il prezzo delle case, il valore di output per una foglia potrebbe essere la media dei prezzi delle case nel gruppo di addestramento che arriva a quella foglia.

Occhio però agli outlier: in alcuni casi potremmo preferire assegnare valori target sulla base di altri indicatori (la mediana, ad esempio).

Una volta assegnati i valori di output a tutte le foglie, l'albero decisionale regressivo è completamente costruito. Quando viene presentata un'istanza di dati al modello, essa percorrerà l'albero fino a raggiungere una foglia, e la predizione sarà il valore di output associato a quella foglia.



Machine learning: Cosa viene dopo?

Nella ultime lezioni, abbiamo visto come risolvere problemi di classificazione e regressione. Ma il machine learning è anche (e soprattutto?) altro...

Sebbene questo sia un corso di fondamenti di intelligenza artificiale, è bene sapere cosa c'è al di là dei modelli di base che abbiamo trattato - anche perché non tutto può essere risolto con i modelli tradizionali.

Considerate questa parte come un **NON argomento di esame**: l'obiettivo è incuriosirvi, non appesantirvi ulteriormente!

Apprendimento
per rinforzo

Oggi come oggi, sentite costantemente parlare di deep learning e, in effetti, i modelli di deep learning sono molto più potenti dei modelli classici.

Deep Learning

Ma non c'è solo il deep learning! L'apprendimento per rinforzo, l'active learning e altro rappresentano dei miglioramenti che possono risolvere problemi più efficientemente dei modelli di deep learning.

Quantum Machine
Learning

Infine, nei prossimi anni la parola "deep learning" verrà probabilmente sostituita con la parola "quantum machine learning"...

Apprendimento per Rinforzo

La caratteristica principale dell'apprendimento per rinforzo sta nel fatto di ricevere feedback, in forma di ricompense (reward), durante l'esecuzione dell'agente.

L'idea del rinforzo sta nell'utilizzo di tecniche di esplorazione dell'ambiente per il miglioramento della conoscenza già acquisita —> *Cosa ci ricorda tutto questo?*

L'ottimizzazione! L'apprendimento per rinforzo si avvicina all'apprendimento umano poiché non si basa solamente sulle etichette assegnate dal progettista, ma è in grado di ottimizzare tali etichette sulla base di quanto ha imparato.

Nella pratica, ogni azione ha un impatto sull'ambiente e ogni ambiente fornisce una ricompensa che guida l'algoritmo di apprendimento —> se l'algoritmo farà qualcosa di positivo, allora sarà premiato; se sbaglierà, allora la sua ricompensa sarà minore, cosicché possa capire il suo errore ed aggiornare la sua conoscenza.

Apprendimento per rinforzo: Task in cui l'obiettivo è imparare i comportamenti corretti partendo da una conoscenza pregressa ma migliorandola iterativamente tramite interazioni con l'ambiente e ottimizzazione di una funzione reward.

In altri termini, il modello è associato ad una *funzione obiettivo* che deve essere massimizzata. La funzione obiettivo è una rappresentazione numerica delle azioni dell'agente sull'ambiente.

Ma come passiamo dalla funzione obiettivo al miglioramento?

Apprendimento per Rinforzo

Per capirlo, torniamo al problema di decidere se andare a giocare o meno a tennis sulla base delle condizioni climatiche.

Giocare	Meteo	Temperatura	Umidità
NO	Soleggiato	Caldo	Elevata
NO	Soleggiato	Caldo	Elevata
SI	Nuvoloso	Caldo	Elevata
SI	Piovoso	Mite	Elevata
SI	Piovoso	Freddo	Normale
NO	Piovoso	Freddo	Normale
SI	Nuvoloso	Freddo	Normale
NO	Soleggiato	Mite	Elevata
SI	Soleggiato	Freddo	Normale
SI	Piovoso	Mite	Normale
SI	Soleggiato	Mite	Normale
SI	Nuvoloso	Mite	Elevata
SI	Nuvoloso	Caldo	Normale
NO	Piovoso	Mite	Elevata

Al seguente input:

Meteo = Pioggia; Temperatura = Caldo; Umidità = Elevata.

un semplice classificatore bayesiano risponderebbe con “NO”.

Ma se, invece, la risposta corretta fosse “SI”, allora potremmo assegnare un punteggio negativo al comportamento del modello:

reward = -10;

Che implicazioni ha questo punteggio?

La cosa più semplice da pensare sarebbe quella di modificare l’etichetta da “NO” a “SI” nei casi in cui il dataset ha, come predittori, gli stessi che hanno causato l’errore...

Ma possiamo subito renderci conto che questa non è un’opzione valida, per due valide ragioni.

Apprendimento per Rinforzo

Senza entrare troppo nel dettaglio, potremmo associare ad ogni azione una probabilità che indichi quanto questa possa portare ad un miglioramento della ricompensa.

Giocare	Meteo	Temperatura	Umidità
NO	Soleggiato	Caldo	Elevata
NO	Soleggiato	Caldo	Elevata
SI	Nuvoloso	Caldo	Elevata
SI	Piovoso	Mite	Elevata
SI	Piovoso	Freddo	Normale
NO	Piovoso	Freddo	Normale
SI	Nuvoloso	Freddo	Normale
NO	Soleggiato	Mite	Elevata
SI	Soleggiato	Freddo	Normale
SI	Piovoso	Mite	Normale
SI	Soleggiato	Mite	Normale
SI	Nuvoloso	Mite	Elevata
SI	Nuvoloso	Caldo	Normale
NO	Piovoso	Mite	Elevata

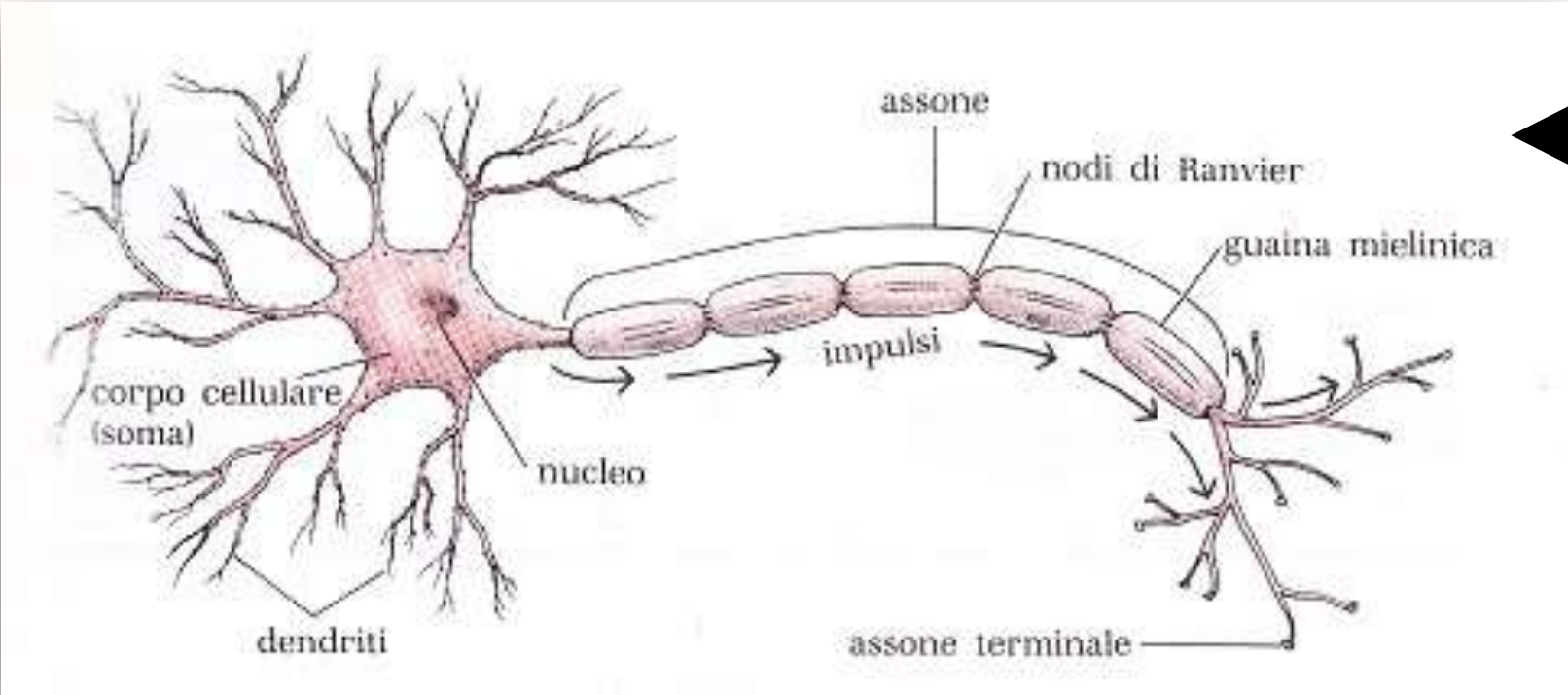
Sulla base di questa informazione, potremmo poi definire una *politica di azione*, ovvero una strategia di aggiornamento delle probabilità e delle azioni che l'agente dovrà effettuare.

Ad esempio, una possibile politica di azione prevede l'utilizzo degli algoritmi greedy (ricordate?), cosicché l'agente effettui l'azione che massimizzi, in un dato momento, la probabilità di reward.

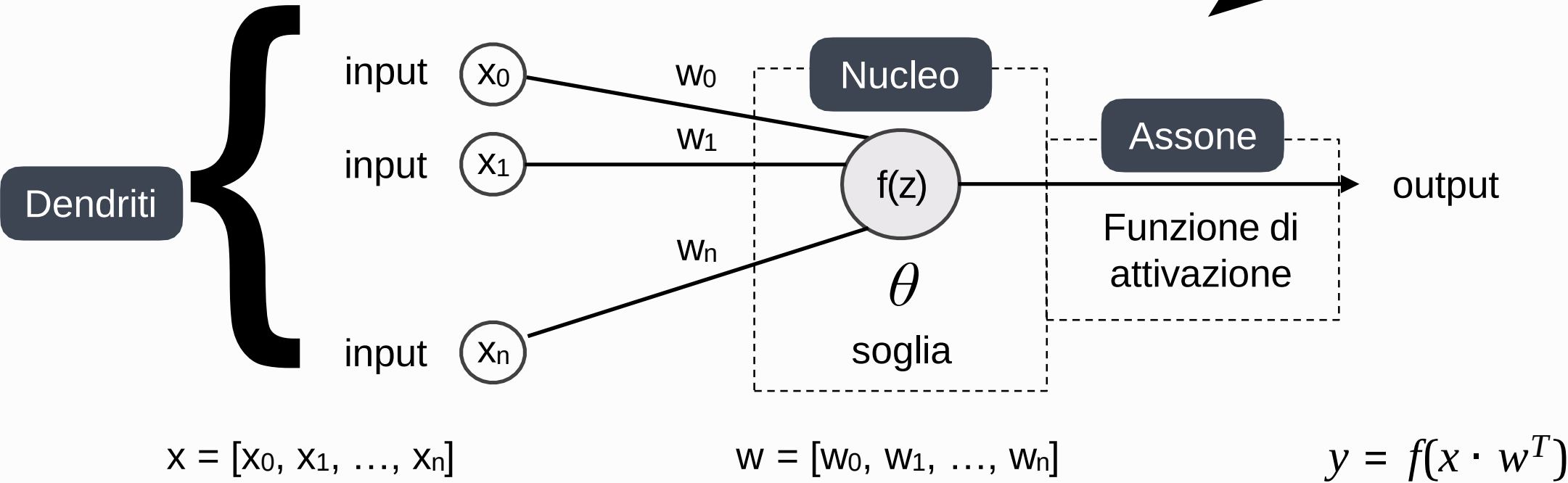
L'aggiornamento delle probabilità è spesso effettuato tramite i cosiddetti processi decisionali di Markov, i metodi di Monte Carlo o attraverso i metodi statistici di apprendimento differenziale temporale.

Questi modelli hanno un costo notevolmente maggiore a quelli di base, ma sono di norma più efficaci ed adattivi.

Deep Learning

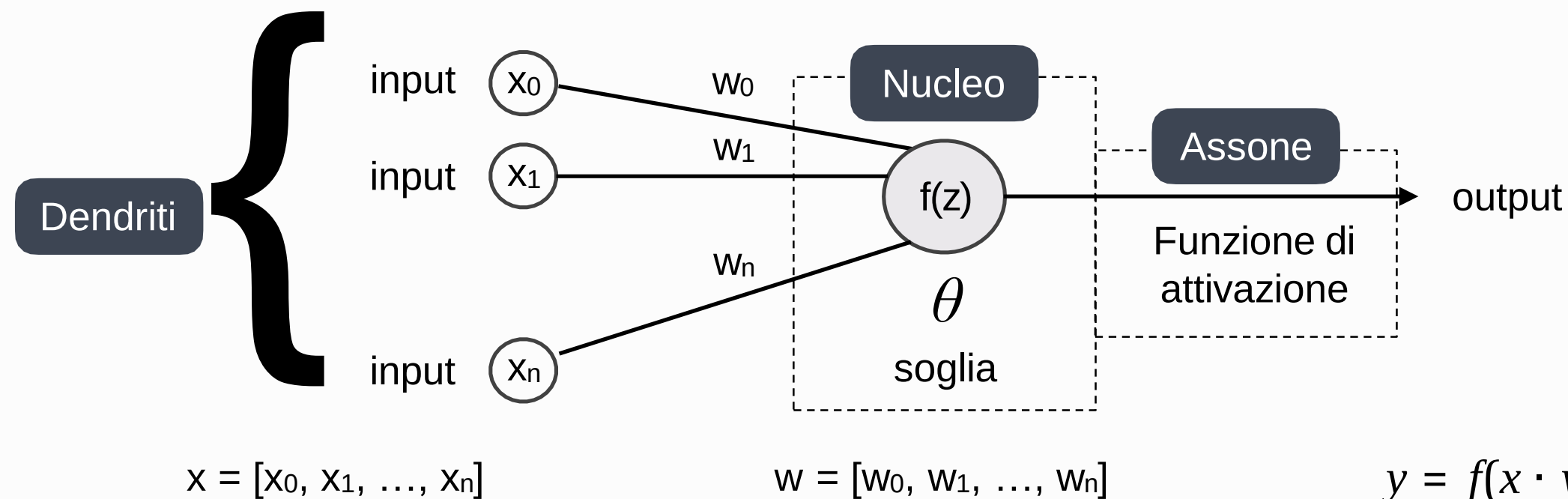


Ricordate il neurone?



Ecco un neurone artificiale

Deep Learning



Se $f(z) > \theta \Rightarrow f(z) - \theta > 0$. In questo caso, il neurone è attivato ed il suo segnale verrà propagato

Una rete di questo tipo è chiamata *rete neurale artificiale* e rappresenta la base per rappresentare reti ancora più complesse.

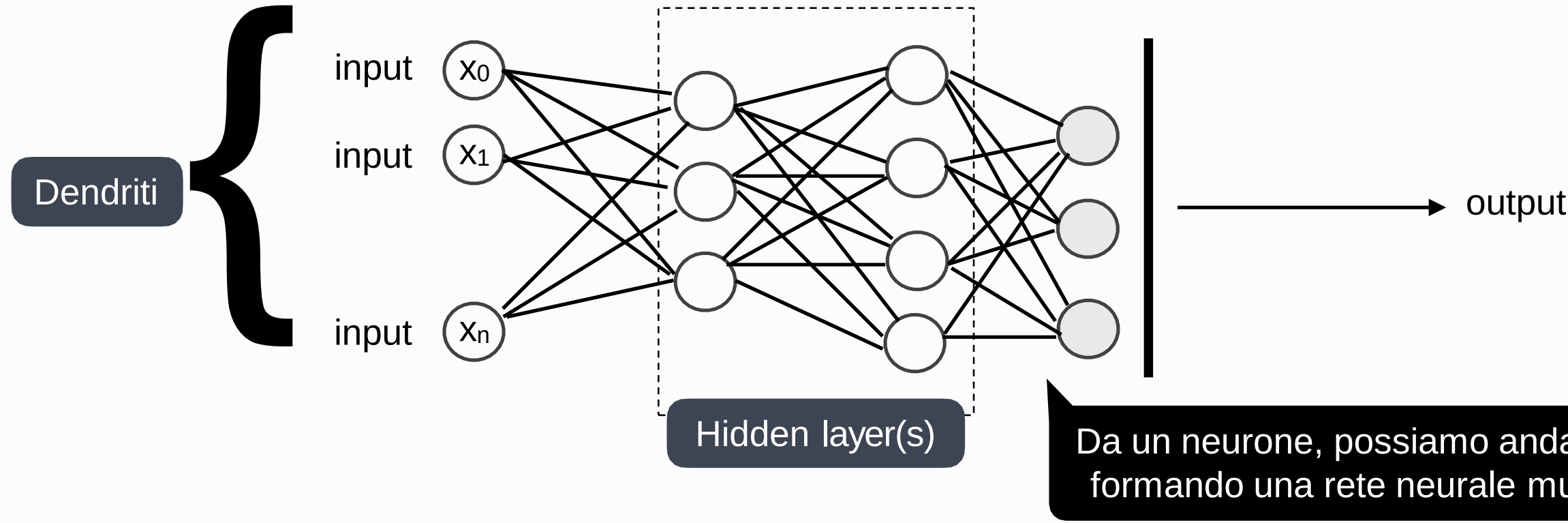
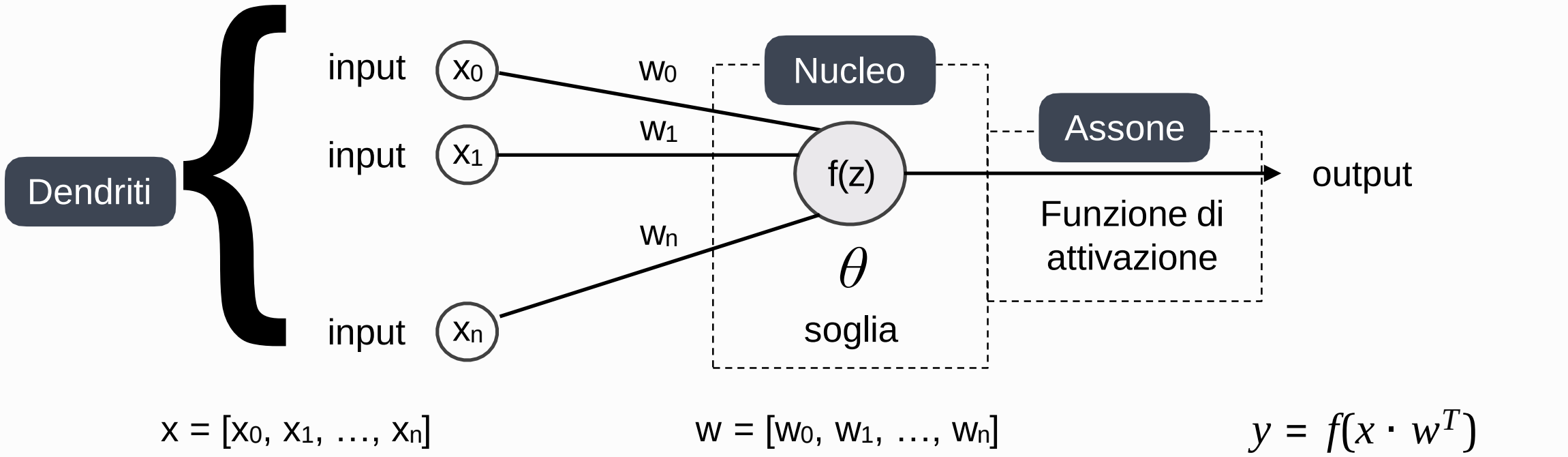
Ma perché dovremmo usare una rete neurale se abbiamo classificatori o regressori più semplici ed intuitivi?

Già guardando la forma del neurone artificiale, possiamo renderci conto che questo **NON presenta** alcuna informazione su ciò che dobbiamo predire —> un tipico caso d'uso è rappresentato dai problemi per cui non conosciamo nulla sulla variabile di output.

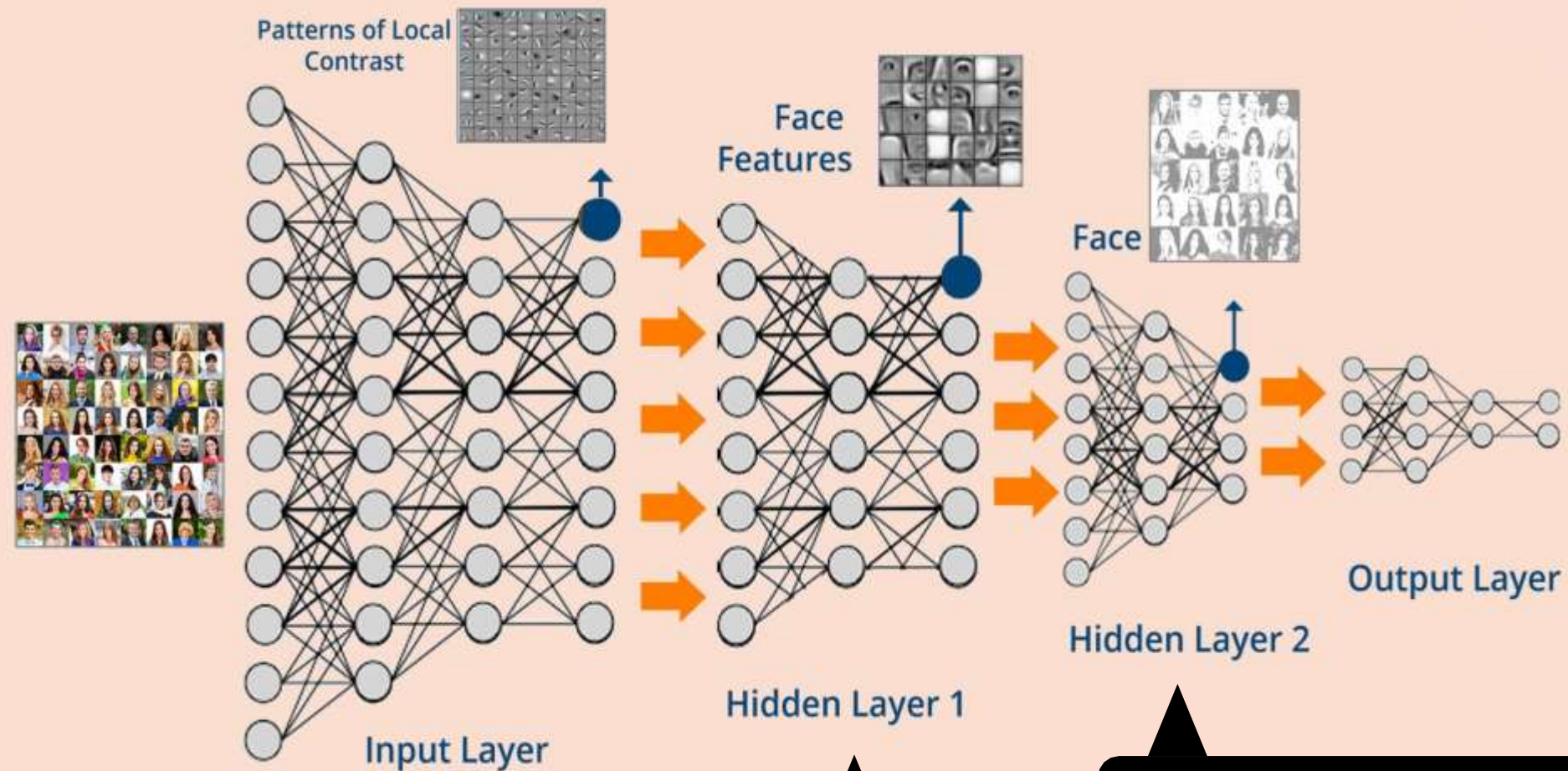
L'esempio classico dell'auto a guida autonoma: come potremmo mai utilizzare un decision tree o un classificatore bayesiano?

Regressione, Regressori, e Oltre

Deep Learning



Deep Learning



Ogni hidden layer sarà specializzato in qualcosa

Man mano che le operazioni andranno avanti, il deep learner avrà le "idee più chiare"

I più interessati, possono dare un'occhiata qui: <https://www.slideshare.net/ashraybhandare/deep-learning-cnn-and-rnn>.

Quantum Machine Learning

“The dream has come true”, dice qualcuno... la tecnologia quantum è ormai dietro le porte e presto potremo sfruttare a pieno le potenzialità dei quantum computer. Alcuni, addirittura, già nominano il 21° secolo come la *“quantum era”*.

Il quantum computing si basa su due idee di base: (1) *superposition*, i q-bit possono assumere stati diversi allo stesso momento; (2) *entanglement*, i q-bit possono essere fortemente connessi anche in assenza di interazioni fisiche dirette.

Oggi come oggi, siamo in attesa della cosiddetta *quantum supremacy*, ovvero il momento in cui un programma interamente su quantum computing sarà in grado di risolvere problemi che nessun altro programma tradizionale può risolvere in nessun ragionevole ammontare di tempo.

Il machine learning rappresenta un problema particolarmente interessante per il quantum computing. Un motivo su tutti: la fase di addestramento è molto costosa, soprattutto quando parliamo di deep learning.

E quindi, possiamo usare delle componenti quantistiche per velocizzare le attività di addestramento dei modelli di machine learning?

Il *quantum-enhanced machine learning* è la branca che si occupa di come ingegnerizzare architetture quantistiche che possano supportare l'addestramento dei modelli di machine learning. Non entreremo nel dettaglio, chiaramente...

Ma è vero anche il contrario...

Regressione, Regressori, e Oltre

Quantum Machine Learning

Quantum machine learning è anche associato all'utilizzo di algoritmi di machine learning che analizzano e predicono dati quantistici.

Sebbene tutto questo sia ancora qualcosa di molto preliminare, la vera sfida è rappresentata dalla possibilità di combinare componenti tradizionali e quantistiche per l'analisi dei dati o, addirittura, creare modelli completamente basati su q-bit (il cosiddetto *fully quantum machine learning*).

Come ultima nota, vale la pena notare che la tecnologia quantistica è usata anche per i problemi di ricerca: ad esempio, un noto algoritmo è il *quantum annealing*.



Qiskit



Cirq



QuantuMoonlight

